

Aislamiento de Datos por Mecánico Independiente

VERSIÓN 2.4

Implementación del modelo de gobernanza multi-inquilino a nivel de aplicación

1. Justificación y Requerimiento Técnico

En la planta industrial de **CottonKnit**, cada mecánico técnico opera de forma autónoma gestionando un inventario propio de máquinas asignadas, diagnósticos de mantenimiento correctivo (FO-026), inspecciones técnicas preventivas (IND-MEC-01 / FO525) y control de agujas. Previamente, el sistema exponía todas las máquinas de la base de datos a cualquier usuario que ingresara. El requerimiento estipuló que **todo nuevo usuario mecánico (ej. Yofre) debe iniciar con sus módulos completamente limpios (0 registros)**, registrando exclusivamente sus propios activos, mientras que el perfil **ADMIN** mantiene una vista global supervisora.

MECANICO

Comportamiento para Mecánicos (ej. Yofre)

- Inicio en Cero:** Tablas de Máquinas, FO-026, Inspecciones y Agujas inician vacías (0 registros).
- Propiedad Exclusiva:** Al crear una máquina o registro, se asocia automáticamente a su `usuario_id`.
- Filtro Automático:** Solo recupera y visualiza registros creados por su cuenta.
- Autocompletado:** Firma y código técnico (MEC-ID) asignados automáticamente en reportes.

ADMIN

Comportamiento para Administrador

- Supervisión Global:** Visualiza la totalidad de máquinas e informes de todos los mecánicos.
- Trazabilidad en Planta:** Nueva columna *Mecánico Responsable* en el Maestro de Máquinas.
- Compatibilidad Retroactiva:** Mantiene acceso a los activos precargados del sistema.
- Gestión de Cuentas:** Control exclusivo para crear, editar o inhabilitar mecánicos.

2. Diagrama de Flujo de Datos y Aislamiento

LOGIN: YOFRE

Token de Sesión + ID: 3



Frontend ApiService

GET /api/maquinas?userId=3



MySQL 8.0

WHERE usuario_id = 3

El mecánico Yofre recibe **0 resultados** hasta que registra sus propias máquinas.

3. Modificaciones en el Esquema de Base de Datos

ENTIDAD / TABLA	CAMPO AGREGADO	TIPO / RESTRICCIÓN	PROPÓSITO DEL CAMBIO
Maquina (maquinas)	usuario_id	INT, NULL, FK usuarios(id)	Asigna la máquina al mecánico propietario. Se removió restricción única global de código.
InspeccionMaquina (inspeccion_maquinas)	usuario_id	INT, NULL, FK usuarios(id)	Asocia las inspecciones IND-MEC-01 al técnico ejecutor.
StockAgujas (stock_agujas)	usuario_id	INT, NULL, FK usuarios(id)	Aísla el conteo físico de agujas por responsable de línea.
ReporteMantenimiento	usuario_id	INT, NULL, FK usuarios(id)	Ya contaba con la relación; se habilitó el filtrado en repositorio y controller.

4. Capa Backend (Spring Data JPA & REST Controllers)

Se extendieron los repositorios con consultas derivadas de Spring Data para filtrar registros por `usuarioId`, y se modificaron los controladores REST para aceptar el parámetro opcional `@RequestParam(required = false) Integer userId`:

```
// MaquinaController.java @GetMapping public List<Maquina> listar(@RequestParam(required = false) Integer userId) { if (userId != null) { return repository.findByUsuarioId(userId); } return repository.findAll(); } // ADMIN: devuelve todo } @PostMapping public ResponseEntity<?> guardar(@RequestBody Maquina maquina, @RequestParam(required = false) Integer userId) { if (maquina.getUsuario() == null && userId != null) { Usuario u = new Usuario(); u.setId(userId); maquina.setUsuario(u); } if (maquina.getId() == null && maquina.getUsuario() != null) { if (repository.existsByCodigoAndUsuarioId(maquina.getCodigo(), maquina.getUsuario().getId())) { return ResponseEntity.badRequest().body("El código ya existe para tu usuario."); } } return ResponseEntity.ok(repository.save(maquina)); }
```

5. Capa Frontend (Angular 21 - ApiService Centralizado)

Toda la lógica de consulta y guardado se abstraigo dentro de `api.service.ts`. Mediante la inyección de `AuthService`, el cliente intercepta si el usuario en sesión es **ADMIN** o **MECANICO**:

```
// api.service.ts private getScopedHttpParams(): HttpParams { let params = new HttpParams(); const user = this.authService.getCurrentUser(); if (user && user.rol !== 'ADMIN') { params = params.set('userId', user.id.toString()); } return params; } // Al listar máquinas, reportes FO-026 o inspecciones: listarMaquinas(): Observable<any[]> { return this.http.get<any[]>(`${this.apiUrl}/maquinas`, { params: this.getScopedHttpParams() }); }
```

6. Validaciones y Pruebas de Compilación Realizadas

✓ **Backend Spring Boot:** Compilación limpia con Maven (`mvn compile -DskipTests`). 34 archivos compilados con Java 21 sin advertencias ni errores de sintaxis.

✓ **Frontend Angular 21:** Generación completa de bundle con `ng build`. Rutas estáticas prerenderizadas y paquetes de distribución optimizados en 7.35s.

7. Procedimiento de Actualización en Servidor de Producción

Los cambios fueron integrados en la rama `main` del repositorio remoto `cbenner2014/Sistema-Documentos-CK`. Para desplegar la actualización en el servidor Docker:

```
# 1. Ingresar al directorio del proyecto en el servidor cd ~/cottonknit # 2. Descargar las últimas modificaciones desde GitHub git pull origin main # 3. Reconstruir las imágenes de Backend y Frontend sin detener la base de datos docker compose up -d --build
```

Resultado Comprobado

Al iniciar sesión en <https://ck.dabecode.com> con el usuario de **Yofre**, el sistema arranca con **0 máquinas y 0 reportes**. Una vez que Yofre registra máquinas o tareas, solo él puede verlas y modificarlas. Al ingresar como **admin**, se observan todos los activos de la planta etiquetados por su respectivo mecánico.